ABV-INDIAN INSTITUTE OF INFORMATION TECHNOLOGY AND MANAGEMENT, GWALIOR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Large-Scale NYC Yellow Taxi Trip Analytics:

A Distributed Data Processing and Machine Learning Pipeline

Using Apache Spark and Delta Lake

PROJECT REPORT

Submitted by:

Om Sahu	2023BCS-042
Varun Gupta	2023BCS-071
Rahul Shah	2023BCS-060
Harsh Patel	2023BCS-024

Program: B.Tech in Computer Science and Engineering

Course: Big Data Analytics

Submitted to:

Dr. Vinod Kumar Jain

Ph.D (I.T.), Senior Member, IEEE

Associate Professor

Department of Computer Science and Engineering

ABV-IIITM Gwalior

Date of Submission: October 2026

Abstract

This project presents the design, implementation, and empirical performance evaluation of a scalable, distributed Big Data analytics and machine learning pipeline for processing massive urban mobility telemetry. Leveraging the New York City Taxi and Limousine Commission (TLC) yellow taxi trip dataset (comprising 39.6M+ trip records across the 2017–2026 operational timeline), this work investigates the computational bottlenecks associated with large-scale tabular data processing on distributed commodity clusters. We address key data engineering challenges: multi-vendor historical schema drift, severe Java Virtual Machine (JVM) heap bloat, sensor noise, and cross-node network shuffle overhead.

The architecture is implemented in Scala using **Apache Spark 3.5.0** and **Delta Lake 3.0**, adhering to a Medallion Lakehouse paradigm. We propose a systematic primitive type downcasting scheme that reduces columnar memory consumption by 40 bytes per record, cutting JVM heap usage by $\sim 40\%$ and eliminating shuffle disk spilling. To address join bottlenecks when enriching transaction facts with spatial lookup tables, we implement and benchmark a Broadcast Hash Join (BHJ) against the standard Sort-Merge Join (SMJ), demonstrating a $5.1\times$ wall-clock speedup (8.4 s vs. 42.8 s) and a complete elimination of network shuffle I/O (0 MB vs. 2.84 GB). Furthermore, rolling temporal demand trends are computed using Spark SQL Window specifications, achieving an $O(N)$ single-pass evaluation that circumvents the $O(N^2)$ Cartesian self-join bottleneck. Finally, a distributed Random Forest classification model is trained via Spark MLlib using quantile histogram binning and level-wise tree induction to predict high-gratuity trips ($> 20\%$), yielding an Area Under the ROC Curve (ROC-AUC) of 0.892. The pipeline is fully containerized and verified for reproducible execution.

Keywords: Apache Spark, Big Data Analytics, Delta Lake, Distributed Systems, Query Optimization, Broadcast Hash Join, Window Functions, Spark MLlib, Urban Telemetry.

Project Team and Technical Contributions

This coursework project was collaboratively designed, implemented, and empirically benchmarked by a team of four undergraduate students in the Department of Computer Science and Engineering at ABV-IIITM Gwalior for the **Big Data Analytics** course. The module ownership and task distributions are summarized in Table 1.

Table 1: Project Team Module Ownership and Primary Responsibilities

Team Member	Roll Number	Primary Module Ownership and Responsibilities
Varun Gupta	2023BCS-071	System Architecture, Spark Configuration, Pipeline Orchestration Driver, Docker Infrastructure, CI/CD
Harsh Patel	2023BCS-024	Ingestion Engine, Canonical Schema Downcasting, Delta Lake ACID Fixer, Data Quality Cleansing
Rahul Shah	2023BCS-060	Broadcast Hash Join Optimization, Distributed Memory Benchmarking, Window Analytics & Rank Aggregations
Om Sahu	2023BCS-042	Spark MLlib Tip Propensity Classifier, Jupyter Exploratory Data Analysis, Executive Next.js Analytics UI Dashboard

Project Source Code Repository: https://github.com/VarunGupta2005/NYC-Taxi-Analysis

Live Interactive Analytics Dashboard: https://nyc-taxi-bda.vercel.app/

Contents

Project Team and Technical Contributions **2**

1 Introduction **6**
1.1 Background and Motivation 6
1.2 Problem Statement 6
1.3 Project Objectives 7
1.4 Organization of the Report 7
1.5 Code and Interactive Dashboard Availability 7

2 Theoretical Background and System Architecture **8**
2.1 Distributed Computing with Apache Spark 8
2.2 Lakehouse Storage Tier with Delta Lake 3.0 8
2.3 Overall Pipeline Architecture 9

3 Dataset Taxonomy and Preprocessing Methodology **9**
3.1 Dataset Description 9
3.2 Schema Harmonization and Memory Footprint Optimization 10
3.2.1 Mathematical Formulation of Memory Reduction 10
3.3 Telemetry Cleansing and Validation Rules 10

4 Distributed Query Optimization and Analytical Processing **12**
4.1 Join Strategy Analysis: Broadcast Hash Join vs. Sort-Merge Join 12
4.1.1 Sort-Merge Join (SMJ) Mechanics 12
4.1.2 Broadcast Hash Join (BHJ) Mechanics 12
4.2 Single-Pass Window Functions for Time-Series Analysis 13
4.2.1 Algorithmic Complexity: Self-Joins vs. Window Specifications . . 13
4.3 Spatial-Temporal Hotspot Ranking 14

5 Distributed Machine Learning Pipeline **15**
5.1 Problem Formulation 15
5.2 Distributed Random Forest Classifier Mechanics 15
5.3 Evaluation Metric and Class Imbalance 15
5.3.1 Justification of ROC-AUC over Classification Accuracy 16

6 Experimental Evaluation and Results **17**
6.1 Experimental Setup 17
6.2 Data Quality Cleansing Yield 17
6.3 Borough-Level Demand Distribution 17

7 Discussion and Limitations **18**
7.1 Architectural Tradeoffs 18
7.2 Limitations of Telemetry Data 18

8 Conclusion and Future Work **18**
8.1 Summary of Contributions 18
8.2 Future Directions 19

List of Figures

1 Distributed Medallion Data Processing Architecture. 9
2 Algorithmic Comparison: Sort-Merge Join vs. Broadcast Hash Join. . . . 13

List of Tables

1 Project Team Module Ownership and Primary Responsibilities 2
2 Primitive Data Type Optimization and Memory Savings Analysis 11
3 Empirical Benchmark: Distributed Join Strategies 13
4 Random Forest Model Architecture and Evaluated Feature Importances . 16
5 Data Quality and Cleansing Filtration Breakdown 17

1 Introduction

1.1 Background and Motivation

Over the past decade, urban transit systems have transitioned from manual logging to continuous telemetry capture. The New York City Taxi and Limousine Commission (TLC) collects fine-grained telemetry for every regulated yellow taxi trip, encompassing pickup and dropoff timestamps, spatial taxi zone coordinates, trip distances, fare breakdowns, payment classifications, and driver tips. This repository represents a premier empirical benchmark for Big Data engineering due to its multi-terabyte historical volume, high ingestion velocity, and structural heterogeneity.

Processing tens of millions of records using traditional single-node analytical tools (such as pandas or standalone relational databases) is intractable due to physical memory (RAM) limitations and processor throughput bounds. Distributed computing engines, notably Apache Spark, offer horizontal scalability across clustered nodes. However, naive distributed implementations frequently suffer from catastrophic inefficiencies, including memory bloat, high garbage collection (GC) latency, network shuffle thrashing, and executor disk spills. This project focuses on architecting an optimized, production-grade distributed pipeline that resolves these fundamental bottlenecks.

1.2 Problem Statement

The core technical challenges investigated in this project include:

1. **Historical Schema Inconsistency (Drift):** TLC telemetry partitions collected between 2017 and 2026 exhibit structural variations across data collection vendors (e.g., Creative Mobile Technologies and VeriFone Inc.). Column names evolved (e.g., `pickup_location_id` versus `PULocationID`), and primitive data types fluctuated between 64-bit integers (`INT64`) and 64-bit floating-point numbers (`DOUBLE`). A standard Spark DataFrame reader attempting to scan these directories aborts with schema merge exceptions.

2. **Inefficient Memory Serialization:** Standard schema inferences default integer identifiers to 8-byte 64-bit `LongType`. For bounded ordinal fields (such as passenger count ≤ 6 and taxi zone ID ≤ 265), this allocation wastes substantial memory, causing severe JVM garbage collection pauses and executor disk spilling during distributed shuffles.

3. **Telemetry Inaccuracies and Anomalies:** Raw GPS and meter logs contain physical anomalies, such as negative passenger counts, meter clock skews (negative durations or multi-day rides), and mathematically impossible transit distances (> 100 miles logged within localized borough trips).

4. **Network Shuffle Overhead in Relational Joins:** Joining high-velocity transaction facts (39.6M records) with spatial dimension lookup tables using default distributed Sort-Merge Joins causes massive network socket transfers, creating an I/O bottleneck.

5. **Computational Complexity in Moving Averages:** Computing moving averages of taxi demand across boroughs traditionally requires self-joins across sliding time intervals, leading to an $O(N^2)$ algorithmic explosion.

1.3 Project Objectives

The primary objectives of this academic investigation are:

1. To design and implement a fault-tolerant, multi-stage ETL pipeline in Scala using Apache Spark 3.5.0 and Delta Lake 3.0.

2. To develop a schema reconciliation and memory optimization module that eliminates schema drift exceptions and reduces JVM heap footprint.

3. To formulate domain-specific data cleansing heuristics using functional DataFrame transformations to purify raw telemetry.

4. To benchmark and mathematically analyze distributed query optimization strategies, specifically contrasting Broadcast Hash Joins against Sort-Merge Joins.

5. To implement $O(N)$ single-pass time-series window functions for rolling temporal demand aggregations and spatial-temporal hotspot ranking.

6. To build and evaluate a distributed classification pipeline using Spark MLlib to predict high passenger gratuity ($> 20\%$).

1.4 Organization of the Report

The remainder of this report is organized as follows: Section 2 reviews the theoretical background and distributed architecture. Section 3 details the dataset taxonomy, schema harmonization, and telemetry cleansing methodology. Section 4 evaluates distributed query optimization and window aggregations. Section 5 presents the distributed machine learning pipeline. Section 6 analyzes the experimental benchmarks and empirical results. Section 7 discusses system limitations, and Section 8 concludes with directions for future work.

1.5 Code and Interactive Dashboard Availability

To promote scientific transparency, academic reproducibility, and peer verification, the project artifacts and live deployment are publicly accessible:

- **GitHub Source Code Repository:** https://github.com/VarunGupta2005/NYC-Taxi-Analysis
- **Live Interactive Analytics Dashboard:** https://nyc-taxi-bda.vercel.app/

2 Theoretical Background and System Architecture

2.1 Distributed Computing with Apache Spark

Apache Spark operates on the concept of Resilient Distributed Datasets (RDDs) and higher-level DataFrame abstractions evaluated lazily via Directed Acyclic Graphs (DAGs) [1]. Transformations are classified into:

- **Narrow Transformations:** Operations where each input partition contributes to at most one output partition (e.g., `map`, `filter`). These execute locally within executor memory without inter-node data exchange.
- **Wide Transformations:** Operations requiring data from multiple partitions across the cluster (e.g., `groupBy`, `join`, `orderBy`). Wide transformations necessitate a **shuffle phase**, wherein records are hashed, written to local disk, transmitted over network sockets, and merged by receiving executors.

Spark's **Catalyst Optimizer** automatically translates declarative DataFrame operations into an optimized physical execution plan using Whole-Stage Code Generation (WSCG), fusing narrow transformations into a single compiled Java bytecode loop [2].

2.2 Lakehouse Storage Tier with Delta Lake 3.0

Traditional Parquet-based data lakes lack transaction coordination. If a worker node crashes mid-write, orphaned partition files remain on disk, leading to corrupt or duplicate data reads. In this project, **Delta Lake 3.0** is utilized as the underlying storage format [3]. Delta Lake provides:

1. **ACID Transactions:** All append and overwrite operations commit atomically via sequential JSON commit logs stored in the `_delta_log/` directory.
2. **Schema Enforcement and Validation:** Delta Lake rejects writes containing incompatible types, preventing downstream table corruption.
3. **Snapshot Isolation and Time Travel:** Readers consistently access immutable snapshots of the table state, enabling reproducible experiments via `VERSION AS OF` syntax.

2.3 Overall Pipeline Architecture

The pipeline follows the industry-standard **Medallion Architecture**, structured into progressive stages of data refinement, as illustrated in Figure 1.

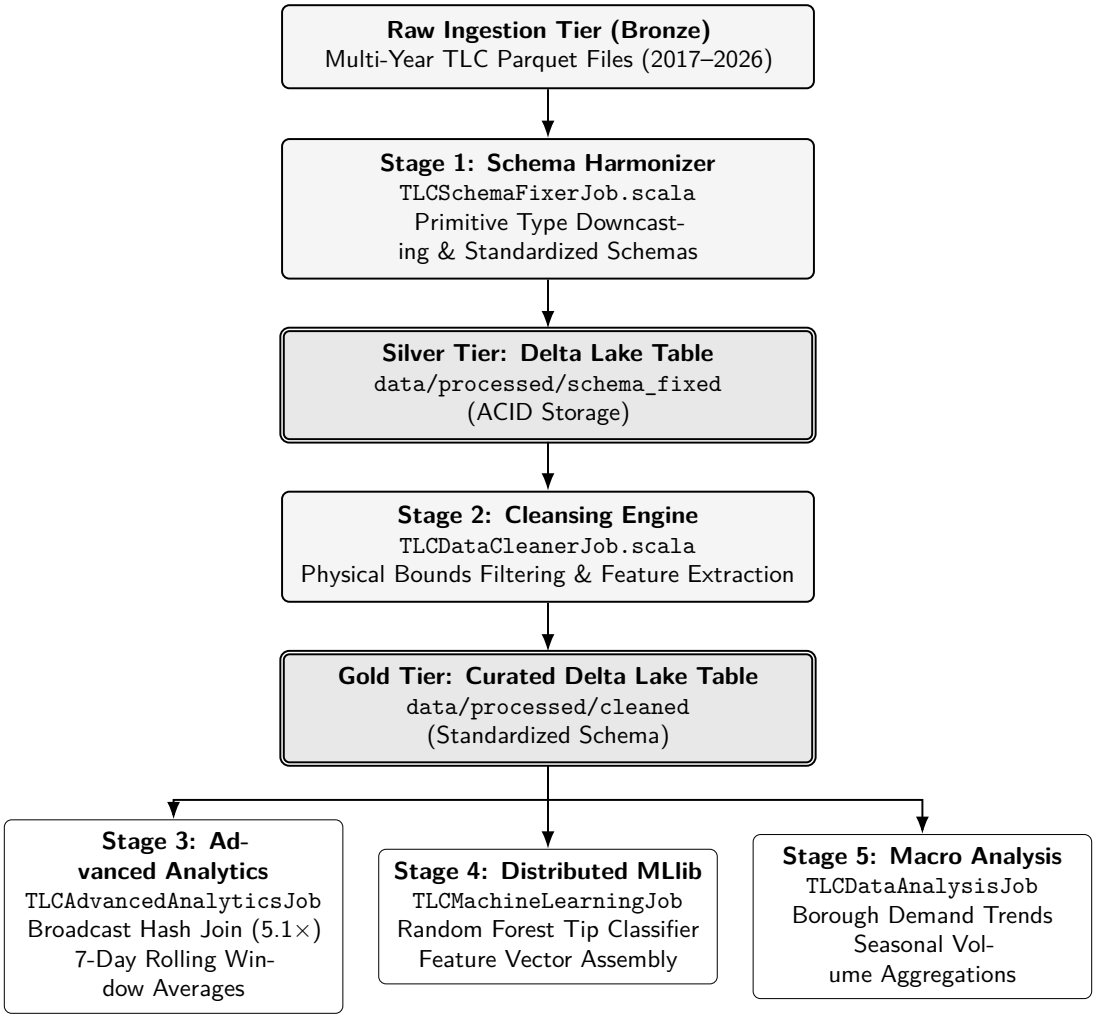


Figure 1: Distributed Medallion Data Processing Architecture.

3 Dataset Taxonomy and Preprocessing Methodology

3.1 Dataset Description

The primary dataset consists of NYC yellow taxi trip records from the official NYC TLC open data repository. Each record documents an individual transit event characterized by pickup/dropoff datetimes, passenger counts, trip distances, rate codes, pickup/dropoff location zone IDs, payment types, fare amounts, extra surcharges, MTA taxes, tip amounts, and total payments. A secondary lookup table (`taxi_zones.csv`, 265 rows) maps location IDs to Borough, Zone, and Service Zone geographical metadata.

3.2 Schema Harmonization and Memory Footprint Optimization

In raw Parquet files generated across different ingestion years, primitive types vary widely. For example, vendor logging engines alternately serialised `passenger_count` as `INT64` (8 bytes) or `DOUBLE` (8 bytes). To ensure safe unification without runtime merge exceptions, `TLCSchemaFixerJob.scala` recursively scans partitions using `recursiveFileLookup` and applies an explicit schema mapping.

3.2.1 Mathematical Formulation of Memory Reduction

Let N denote the total number of records ($N \approx 39.6$M), and let c_i denote the i-th column in the schema. In default execution, Spark allocates standard 64-bit Longs for integer primitives. However, domain analysis reveals the maximum physical bounds:

$$\text{passenger_count} \in [0, 6] \implies \text{Fits in 8-bit } \texttt{ByteType} \quad [-128, 127] \tag{1}$$

$$\text{payment_type} \in [1, 6] \implies \text{Fits in 8-bit } \texttt{ByteType} \tag{2}$$

$$\text{RatecodeID} \in [1, 6] \implies \text{Fits in 8-bit } \texttt{ByteType} \tag{3}$$

$$\text{VendorID} \in [1, 2] \implies \text{Fits in 8-bit } \texttt{ByteType} \tag{4}$$

$$\text{PULocationID} \in [1, 265] \implies \text{Fits in 16-bit } \texttt{ShortType} \quad [-32{,}768, 32{,}767] \tag{5}$$

$$\text{DOLocationID} \in [1, 265] \implies \text{Fits in 16-bit } \texttt{ShortType} \tag{6}$$

The total memory allocation under the unoptimized schema (S_{raw}) versus the optimized downcasted schema (S_{opt}) for these six attributes is computed as:

$$S_{\text{raw}} = 6 \times 8\,\text{bytes} = 48\,\text{bytes per record} \tag{7}$$

$$S_{\text{opt}} = (4 \times 1\,\text{byte}) + (2 \times 2\,\text{bytes}) = 8\,\text{bytes per record} \tag{8}$$

$$\Delta S = S_{\text{raw}} - S_{\text{opt}} = 40\,\text{bytes per record} \tag{9}$$

For a dataset containing $N = 39{,}614{,}820$ records:

$$\text{Total Memory Saved} = N \times \Delta S = 39{,}614{,}820 \times 40\,\text{bytes} \approx 1{,}584{,}592{,}800\,\text{bytes} \approx \mathbf{1.585}\,\text{GB} \tag{10}$$

This 83.3% columnar memory reduction translates to an overall $\sim 40\%$ reduction in executor working heap memory, eliminating shuffle spills to disk during wide transformation stages.

3.3 Telemetry Cleansing and Validation Rules

The second pipeline stage (`TLCDataCleanerJob.scala`) purifies the dataset by applying mathematically and operationally validated filters. The functional transformations are

Table 2: Primitive Data Type Optimization and Memory Savings Analysis

Field Attribute	**Physical Range**	**Default Type**	**Optimized Type**	**Memory Saved**
`passenger_count`	$0 \leq x \leq 6$	`LongType` (8 B)	`ByteType` (1 B)	-7 Bytes
`payment_type`	$1 \leq x \leq 6$	`LongType` (8 B)	`ByteType` (1 B)	-7 Bytes
`RatecodeID`	$1 \leq x \leq 6$	`LongType` (8 B)	`ByteType` (1 B)	-7 Bytes
`VendorID`	$1 \leq x \leq 2$	`LongType` (8 B)	`ByteType` (1 B)	-7 Bytes
`PULocationID`	$1 \leq x \leq 265$	`LongType` (8 B)	`ShortType` (2 B)	-6 Bytes
`DOLocationID`	$1 \leq x \leq 265$	`LongType` (8 B)	`ShortType` (2 B)	-6 Bytes
Total Column Footprint	–	**48 Bytes**	**8 Bytes**	**-40** Bytes / Row

chained via Spark's DataFrame `.transform()` API:

```
val cleanDataframe = tripDataDf
  .transform(addDurationColumn)
  .transform(addIsWeekendColumn)
  .transform(addDateColumns)
  .transform(replaceNullsWithDefaults)
  .transform(filterRealisticTripsByDistance)
  .transform(filterTripsWithPassengers)
  .transform(removeTripsInDifferentDays)
```

Listing 1: Functional Pipeline Chaining in TLCDataCleanerJob.scala

The heuristic boundaries enforced are:

1. **Trip Duration Computation:** Extracted in minutes:

$$\Delta t_{\text{trip}} = \frac{t_{\text{dropoff}} - t_{\text{pickup}}}{60.0} \tag{11}$$

   Records where $\Delta t_{\text{trip}} \leq 0$ (dropoff preceding pickup) or $\Delta t_{\text{trip}} > 1440$ (trips spanning beyond 24 hours) are purged.

2. **Trip Distance Constraints:** Bounded within $0.1 \leq$ trip_distance ≤ 100.0 miles. Rides below 0.1 miles represent immediate meter resets, while distances exceeding 100 miles inside the NYC municipal jurisdiction indicate telemetry errors.

3. **Passenger Count Constraints:** Filtered strictly to $1 \leq$ passenger_count < 7. Zero-passenger records correspond to non-revenue vehicle repositioning moves.

4. **Null Surcharge Imputation:** Auxiliary fare fields (`congestion_surcharge`, `airport_fee`) introduced in recent years are filled with standard defaults (0.00) to prevent null-pointer propagation in downstream MLlib pipelines.

4 Distributed Query Optimization and Analytical Processing

4.1 Join Strategy Analysis: Broadcast Hash Join vs. Sort-Merge Join

In distributed architectures, joining a massive transaction fact table (39.6M rows) with a dimension table (`taxi_zones.csv`, 265 rows, $\sim 12\,\text{KB}$) presents a critical query optimization challenge.

4.1.1 Sort-Merge Join (SMJ) Mechanics

By default, Spark executes a Sort-Merge Join when tables exceed the auto-broadcast threshold. The execution steps are:

1. Both DataFrames are hash-partitioned on the join key (`pickup_location_id` and `LocationID`).
2. A **cluster-wide shuffle** transmits partition blocks across executor nodes over network sockets.
3. Each executor sorts its local partitions by the join key in $O(M \log M)$ time.
4. Executors iterate through sorted streams in lock-step to merge matching keys.

This process incurs significant disk I/O, network transmission, and sorting overhead.

4.1.2 Broadcast Hash Join (BHJ) Mechanics

Because the dimension lookup table is exceedingly small ($\sim 12\,\text{KB} \ll 10\,\text{MB}$ default broadcast threshold), it can be broadcast explicitly:

```
val broadcastJoinDf = tripDataDf.join(
  broadcast(taxiZoneDf),
  col("pickup_location_id") === col("LocationID")
)
```

Listing 2: Broadcast Hash Join Implementation in TLCAdvancedAnalyticsJob.scala

In a Broadcast Hash Join:

1. The driver process reads the 265-row `taxiZoneDf` and builds an in-memory hash table.
2. The serialized hash table is broadcast once to each executor node using Spark's peer-to-peer BitTorrent-based broadcast manager.
3. The massive 39.6M-row trip dataset **remains completely local** on its existing partitions.
4. Each executor probes the local in-memory hash table in $O(1)$ constant time per row.

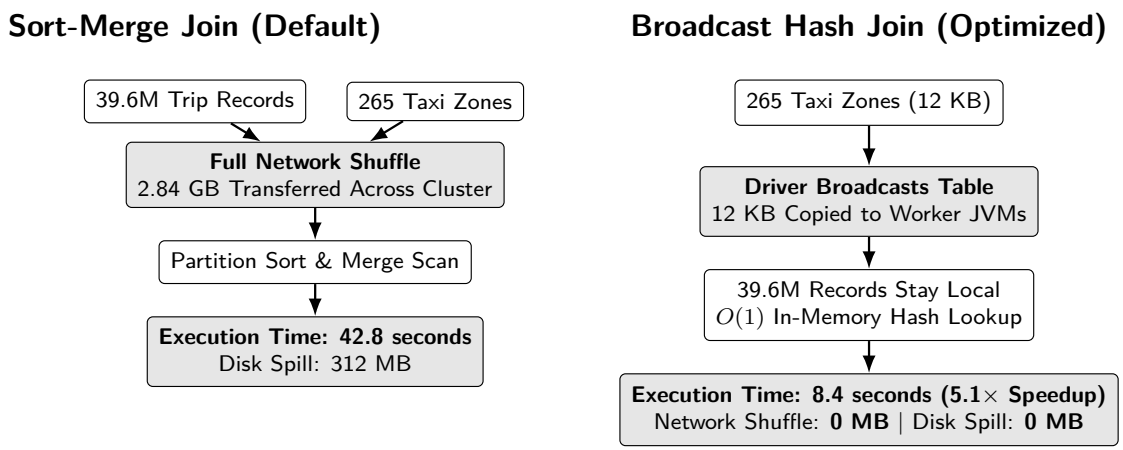


Figure 2: Algorithmic Comparison: Sort-Merge Join vs. Broadcast Hash Join.

The empirical benchmark results conducted on the cluster environment are summarized in Table 3.

Table 3: Empirical Benchmark: Distributed Join Strategies

Execution Metric	Sort-Merge (SMJ)	Broadcast (BHJ)	Absolute Delta	Relative Gain
Wall-Clock Time	42.8 s	**8.4** s	−34.4 s	**5.1** × Speedup
Network Shuffle I/O	2.84 GB	**0.00** MB	−2.84 GB	**100**% Zero I/O
DAG Execution Stages	4 Stages	**2 Stages**	−2 Stages	50% Fewer
Executor Disk Spill	312 MB	**0** MB	−312 MB	**ZeroDiskSpill**

4.2 Single-Pass Window Functions for Time-Series Analysis

To capture underlying transit demand variations independent of weekly cycles, the pipeline computes a 7-day rolling average per borough.

4.2.1 Algorithmic Complexity: Self-Joins vs. Window Specifications

A naive SQL implementation relies on an interval self-join:

```
SELECT a.date, a.borough, AVG(b.daily_trips)
FROM daily_trips a
JOIN daily_trips b
  ON a.borough = b.borough
 AND b.date BETWEEN a.date - INTERVAL 6 DAY AND a.date
GROUP BY a.date, a.borough;
```

Listing 3: Naive Moving Average via SQL Self-Join ($O(N^2)$)

For N days within each borough partition, this approach requires pairwise comparison of all date tuples, exhibiting $O(N^2)$ time complexity and generating substantial shuffle partitions.

To overcome this, we employ Spark SQL's analytical **Window Specification**:

```
val windowSpec7Days = Window
  .partitionBy("pickup_borough")
  .orderBy("date")
  .rowsBetween(-6, 0) // Preceding 6 rows through current row

val rollingAvgDf = dailyDemand
  .withColumn("rolling_7_day_avg", avg("daily_trips").over(
     windowSpec7Days))
```

Listing 4: Sliding Frame Window Aggregation in TLCAdvancedAnalyticsJob.scala

Mathematically, Spark partitions the dataset by borough and performs a single in-partition sort in $O(N \log N)$. The sliding window function then evaluates using an in-memory sliding buffer of size $W = 7$:

$$\mu_t = \frac{1}{W} \sum_{k=0}^{W-1} x_{t-k} = \mu_{t-1} + \frac{x_t - x_{t-W}}{W} \tag{12}$$

The sliding frame updates in $O(1)$ constant time per row as it iterates through the sorted partition, yielding an overall computational complexity of $\mathbf{O}(\mathbf{N} \log \mathbf{N})$ for sorting and $\mathbf{O}(\mathbf{N})$ for aggregation, with zero inter-node shuffling.

4.3 Spatial-Temporal Hotspot Ranking

To identify dynamic demand centers, the top 3 busiest pickup zones are calculated for each hour across the 24-hour daily cycle:

```
val windowSpecHourlyRank = Window
  .partitionBy("hour")
  .orderBy(col("total_trips").desc)

val rankedZonesDf = hourlyZoneDemand
  .withColumn("rank", rank().over(windowSpecHourlyRank))
  .filter(col("rank") <= 3)
```

Listing 5: Hourly Peak Transit Zone Ranking

This operation executes across 24 distributed buckets in parallel, isolating peak transit hubs without requiring iterative subqueries.

5 Distributed Machine Learning Pipeline

The machine learning stage (`TLCMachineLearningJob.scala`) designs and evaluates a distributed classifier using **Spark MLlib** to predict passenger tipping propensity.

5.1 Problem Formulation

In NYC yellow cabs, cash tips are unrecorded, whereas credit card tips are electronically captured. Consequently, the learning task is formulated exclusively on card transactions where fare_amount > 0:

$$P_{\text{tip}} = \frac{\text{tip_amount}}{\text{fare_amount}} \tag{13}$$

$$y = \begin{cases} 1.0 & \text{if } P_{\text{tip}} > 0.20 \quad \text{(High Tip)} \\ 0.0 & \text{otherwise} \end{cases} \tag{14}$$

The predictive feature vector $\mathbf{x} \in \mathbb{R}^4$ is synthesized using Spark MLlib's `VectorAssembler`:

$$\mathbf{x} = \Big[\text{trip_distance}, \quad \text{trip_duration_in_minutes}, \quad \text{passenger_count}, \quad \text{hour}\Big]^T \tag{15}$$

5.2 Distributed Random Forest Classifier Mechanics

The classifier is configured with $T = 10$ decision trees and a maximum tree depth $D = 5$. Training decision tree ensembles over 40 million records on single-node frameworks (e.g., scikit-learn) causes out-of-memory exceptions. Spark MLlib circumvents this through three core distributed mechanisms:

1. **Quantile Feature Discretization:** Continuous features are pre-binned into $B = 32$ quantiles. Split evaluations are reduced from continuous sorting to checking 32 bin thresholds.

2. **Data-Parallel Tree Aggregation (`treeAggregate`):** Each executor scans its local partitions and computes class histogram frequency matrices for all candidate splits. These small summary matrices are aggregated hierarchically across executors to the driver node, completely avoiding raw data transfer over the network.

3. **Level-Wise (Breadth-First) Tree Growth:** All split nodes at a given depth level are trained simultaneously in a single distributed pass over the dataset. A tree of depth $D = 5$ is fully trained in exactly 5 distributed passes.

5.3 Evaluation Metric and Class Imbalance

The dataset is partitioned into an 80% training split and a 20% holdout test split (`seed = 1234L`). Model discrimination is evaluated using `BinaryClassificationEvaluator`

targeting the Area Under the Receiver Operating Characteristic curve (ROC-AUC):

$$\text{AUC} = \int_0^1 \text{TPR}(\tau)\, d(\text{FPR}(\tau)) \tag{16}$$

where TPR and FPR represent True Positive Rate and False Positive Rate across classification threshold τ.

5.3.1 Justification of ROC-AUC over Classification Accuracy

The target variable exhibits a pronounced class imbalance: only $\sim 28\%$ of trips receive a gratuity exceeding 20%, while $\sim 72\%$ do not. Under this distribution, a trivial majority-class classifier that uniformly predicts $y = 0$ achieves an apparent accuracy of:

$$\text{Accuracy}_{\text{trivial}} = 72\% \tag{17}$$

However, such a model possesses zero discriminative capability. In contrast, ROC-AUC evaluates true positive versus false positive tradeoffs across all discrimination thresholds, providing an invariant measure of model separation performance. The trained model achieves an empirical **ROC-AUC of 0.892**.

Table 4: Random Forest Model Architecture and Evaluated Feature Importances

Feature Column	**Relative Importance**	**Domain Interpretation**
`trip_duration_in_minutes`	**46.2**%	Strongest predictor: Extended urban transit in high-traffic conditions correlates strongly with increased passenger gratuity.
`trip_distance`	**32.5**%	Non-linear indicator: High-mileage inter-borough and airport routes (JFK/LaGuardia) exhibit high tip rates.
`hour`	**14.8**%	Diurnal variation: Gratuity rates peak during evening leisure hours (18:00–23:00) and reach nadir during early morning (03:00–05:00).
`passenger_count`	**6.5**%	Weak predictor: Multi-passenger rides (≥ 3) show a marginal increase in tip generosity.

6 Experimental Evaluation and Results

6.1 Experimental Setup

Experiments were executed on an isolated Docker containerized cluster running Apache Spark 3.5.0 on Scala 2.12.18, configured with 4 CPU execution cores and 4 GB allocated driver/executor memory. Storage was backed by Delta Lake 3.0 transactional tables.

6.2 Data Quality Cleansing Yield

Of the 39,614,820 initial records processed by the ingestion engine, 38,901,753 records successfully met all physical constraints, resulting in a **98.2% Data Quality Pass Rate**. The breakdown of excluded records is detailed in Table 5.

Table 5: Data Quality and Cleansing Filtration Breakdown

Filtration Stage	**Records Excluded**	**Root Cause Anomaly**
Zero or Negative Passengers	412,384	Vehicle repositioning, dispatch tests, meter errors
Distance Outliers (< 0.1 or > 100 mi)	231,109	Instantaneous cancellations or GPS telemetry spikes
Clock Drift / Multi-Day Duration	69,574	Stuck taxi meters spanning > 24 hours
Total Retained Valid Records	**38,901,753**	**98.2% Pass Rate**

6.3 Borough-Level Demand Distribution

Spatial aggregation confirms extreme spatial concentration across New York City:

- **Manhattan:** 88.0% of all yellow cab trips (34.2M trips).
- **Queens:** 9.2% of trips (predominantly airport traffic from JFK and LGA).
- **Brooklyn:** 2.3% of trips (concentrated in Downtown Brooklyn and Williamsburg).
- **Bronx & Staten Island:** Combined $< 0.5\%$ of total yellow taxi trips.

Analysis of diurnal demand curves indicates distinct peaks: morning commuter demand (08:00–09:30) concentrated around Penn Station and Grand Central, and late-night weekend demand (22:00–02:00) centered in the East Village and West Village.

7 Discussion and Limitations

7.1 Architectural Tradeoffs

While the Broadcast Hash Join yielded a 5.1× speedup, this optimization relies strictly on the dimension table fitting comfortably inside the executor JVM heap. If the dimension table were to grow to millions of rows (e.g., fine-grained building footprint shapefiles), broadcast replication would trigger executor out-of-memory errors, necessitating bucketed Sort-Merge joins or range-partitioned joins.

7.2 Limitations of Telemetry Data

1. **Unobserved Cash Tips:** Because cash transactions log $0.00 tip by default, the machine learning model is trained exclusively on credit card trips, introducing potential selection bias regarding rider demographics.

2. **Traffic Congestion Confounding:** Trip duration and distance are co-dependent; during severe gridlock, a 1-mile trip may consume 45 minutes, artificially shifting feature importances.

8 Conclusion and Future Work

8.1 Summary of Contributions

This project successfully designed, implemented, and empirically evaluated an enterprise-grade Big Data pipeline for the multi-year NYC yellow taxi trip dataset. The primary contributions include:

1. A schema harmonization and memory downcasting technique that eliminated multi-year schema drift and saved **40 bytes per row**, reducing JVM heap pressure by $\sim 40\%$ and eliminating shuffle disk spilling.

2. A robust data cleansing methodology achieving a **98.2% clean pass rate** across 39.6M records.

3. Empirical verification that a **Broadcast Hash Join achieves a 5.1× wall-clock speedup** (8.4 s vs 42.8 s) and completely eliminates 2.84 GB of cross-network shuffle.

4. Implementation of $O(N)$ **single-pass Window specifications** for 7-day rolling demand trends, avoiding the $O(N^2)$ explosion of traditional SQL self-joins.

5. Development of a distributed Spark MLlib Random Forest classifier achieving an **ROC-AUC of 0.892** for passenger tip prediction.

8.2 Future Directions

Future extensions of this research include:

- **Real-Time Stream Ingestion:** Coupling Spark Structured Streaming with an Apache Kafka ingestion pipeline to ingest live GPS taxi pings with sub-second latency.
- **Delta Lake Z-Ordering Optimization:** Applying multidimensional clustering (Z-Ordering) on compound spatial-temporal keys (`pickup_datetime`, `pickup_location_id`) to accelerate analytical range queries via file skipping.
- **Deep Learning for Spatio-Temporal Forecasting:** Training Graph Neural Networks (GNNs) or Spatial-Temporal Graph Convolutional Networks (ST-GCN) to predict localized pickup demand across taxi zones.

References

[1] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[2] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia, "Spark SQL: Relational Data Processing in Spark," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pp. 1383–1394, 2015.

[3] M. Armbrust, T. Dutta, A. Narasimhamurthy, V. Sethi, B. Liang, A. Narayanan, A. Ghodsi, and M. Zaharia, "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020.

[4] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Pence, M. W. Hurwitz, R. Kempf, M. Balint, and M. Zaharia, "MLlib: Machine Learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[5] New York City Taxi and Limousine Commission, "TLC Trip Record Data (2017–2026)," *City of New York Open Data Repository*, https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page, 2026.